



Diseño previo del proceso ETL

Curso:

T/6900 Inteligencia de Negocios

Profesor:

Michael Lizandro Sánchez Soto

Estudiantes:

Esteban Josué Brenes Montoya
Joseph Marcell Lugo Brown
Isaac David Elizondo Espinoza

Actividad:

Prueba Corta 3

Fecha de entrega:

29/04/2026

Propósito

Este documento concentra todo el diseño que el equipo definió antes de empezar a programar el ETL. Acá va el stack, el perfilamiento del dataset, las reglas que vamos a aplicar y el esquema final que tienen que producir las dos implementaciones (la manual y la apoyada con IA generativa).

La idea es dejarlo cerrado antes de tocar código. Si después tenemos que cambiar una decisión, se hace acá primero y luego se replica en las dos implementaciones, para que la comparación final sea justa.

1. Stack técnico y reto del equipo

Todo el ETL corre sobre Databricks Free Edition. La elección no fue casual: queríamos un único entorno que cubriera ingesta, transformación, almacenamiento intermedio y carga final sin tener que pegar piezas de servicios distintos. Los trials gratuitos de Snowflake o BigQuery también estaban sobre la mesa, pero los descartamos porque vencen y no queremos depender de eso a mitad de proyecto.

El stack queda así:

Fase ETL	Herramienta	Descripción	¿Familiar?
Extracción / ingesta	Python (pandas)	Lectura del CSV fuente desde almacenamiento local o DBFS Volume.	Sí
Almacenamiento bronze	Delta Lake	Capa raw en Unity Catalog, datos sin transformar con timestamp de carga.	No
Transformación	PySpark + Notebooks	Aplicación de reglas de limpieza sobre Spark DataFrames.	Parcial
Orquestación	Databricks Workflows	Encadenamiento de notebooks con dependencias y manejo de errores.	No (reto)
Carga final cloud	Delta Lake (Silver/Gold)	Capas clean y curada en Unity Catalog con ACID y schema enforcement.	No

El reto principal: Databricks Workflows

De todo el stack, lo que más nos saca de la zona de confort es la orquestación con Databricks Workflows (Jobs). Estamos acostumbrados a correr scripts de Python uno detrás de otro, en orden, a mano. Workflows pide pensar distinto: cada notebook es una tarea, hay dependencias declaradas entre tareas, manejo de fallos, reintentos automáticos y control de cuotas de cómputo serverless.

Aparte de eso, Delta Lake con Unity Catalog también es nuevo para el equipo, sobre todo el schema enforcement y las propiedades ACID. Con pandas y CSVs nunca tuvimos que pensar en eso.

2. Perfilamiento del dataset

Antes de escribir cualquier regla, corrimos un script de exploración sobre el CSV original (12,575 filas, 11 columnas). El objetivo era encontrar todo lo que se pudiera ir mal con los datos: nulos, duplicados, formatos raros, valores fuera de rango, inconsistencias entre columnas.

Lo primero que vale la pena decir es lo que NO está roto. No hay filas duplicadas. No hay Transaction IDs repetidos. Los formatos de Transaction ID y Customer ID se respetan en todas las filas. Las categorías están bien escritas (sin variantes tipográficas). Y cuando las tres medidas numéricas existen, Total Spent siempre coincide con $\text{Price} \times \text{Quantity}$. Hasta ahí, todo bien.

Lo que sí está mal se concentra en los nulos y en algunos tipos de datos. Estos son los hallazgos ordenados por severidad:

#	Columna	Cantidad	% del total	Hallazgo
1	Discount Applied	4,199	33.4 %	El problema más grave del dataset. Un tercio de las transacciones no tiene registro de descuento.
2	Item	1,213	9.6 %	Texto del producto vacío. Sin Item no se puede vincular la transacción a la dimensión de productos.
3	Price Per Unit	609	4.8 %	Precio unitario faltante. La mayoría de estos casos aún tiene Quantity y Total Spent.
4	Quantity	604	4.8 %	Cantidad vendida ausente. Suele coincidir con Total Spent también nulo.
5	Total Spent	604	4.8 %	Monto total faltante. Alta correlación con Quantity nulo.

#	Columna	Cantidad	% del total	Hallazgo
6	Discount Applied	12,575	100 %	Toda la columna llega como string ('True'/'False') en lugar de booleano nativo.
7	(combinaciones)	604	4.8 %	Filas con dos o más medidas numéricas nulas a la vez. Imposibles de reconstruir.
8	Transaction Date	12,575	100 %	Llega como texto, no como fecha. Hay que parsearla para poder usarla en la dimensión de tiempo.
9	Item / Category	—	—	Los sufijos del Item codifican la categoría (BEV, BUT, FOOD, etc.). Sirve como validación cruzada.
10	Total Spent	—	—	Cuando las tres medidas existen, Total Spent siempre coincide con Price × Quantity. Buena consistencia interna.

De acá salen dos observaciones importantes que después condicionan las reglas. Primero, los 609 casos con Price nulo casi siempre tienen Total y Quantity disponibles, así que el precio se puede recuperar matemáticamente. Segundo, los 604 casos con Quantity o Total nulos suelen venir con dos o más nulos a la vez, lo que los vuelve irrecuperables.

Sobre el caso de Discount Applied: el 33% de nulos es demasiado para ignorarlo. Lo conversamos en el equipo y decidimos asumir que un nulo equivale a 'no se aplicó descuento'. Es un supuesto fuerte y vamos a dejarlo documentado como tal en el PDF final.

3. Reglas de transformación

Cada hallazgo del perfilamiento tiene una respuesta concreta. Las reglas las dejamos congeladas acá: ambas implementaciones del ETL las aplican igual, sin variaciones, para que la comparación final no se contamine por decisiones distintas.

Columna / situación	Regla aplicada	Por qué decidimos esto
Item nulo	Eliminar la fila completa.	Sin Item no podemos asignar la venta a la dimensión de productos. Hay 200 ítems posibles, no es algo que se pueda imputar.
Price Per Unit nulo (con Total y Quantity presentes)	Recalcular como Total / Quantity.	Si tenemos los otros dos, el dato existe matemáticamente. Es la

Columna / situación	Regla aplicada	Por qué decidimos esto
		imputación más conservadora posible.
Price Per Unit nulo (sin Total ni Quantity)	Imputar con la mediana del Item.	La mediana por ítem es más representativa que la del Category, porque dentro de una categoría hay rangos de precios distintos.
Quantity nulo (con Total y Price presentes)	Recalcular como $\text{Total} / \text{Price}$ y redondear.	Mismo principio que arriba. Quantity siempre es entero, así que se redondea al entero más cercano.
Quantity o Total nulos sin posibilidad de cálculo	Eliminar la fila.	No tiene sentido inventar una venta. Estimamos que se pierden ~600 filas por esta vía, lo asumimos.
Total Spent (en todos los casos válidos)	Recalcular siempre como $\text{Price} \times \text{Quantity}$.	Garantiza consistencia entre las tres medidas, incluso en filas donde el Total original era correcto.
Discount Applied nulo	Imputar con False.	Asumimos que si el sistema no registró descuento, es porque no se aplicó. Es una decisión que vale la pena documentar como supuesto.
Discount Applied (string 'True'/'False')	Convertir a booleano nativo.	Un BOOLEAN ocupa menos, indexa mejor y simplifica las operaciones de filtrado en la capa Gold.
Transaction Date	Convertir de string a tipo DATE.	Necesario para alinear con dim_date y para poder hacer drill-down por año/mes/trimestre.
Payment Method fuera del dominio	Eliminar la fila.	Solo se aceptan Cash, Credit Card y Digital Wallet. Si aparece otro valor, el registro se considera corrupto.
Location fuera del dominio	Eliminar la fila.	Solo se aceptan Online e In-store. Esta dimensión es clave para el análisis de canal de venta.

Columna / situación	Regla aplicada	Por qué decidimos esto
Validaciones preventivas (formato Transaction ID, Customer ID, valores ≤ 0)	Mantener las reglas aunque no se detecten casos.	En el dataset actual no hay violaciones, pero las reglas se conservan para futuras cargas y como salvaguarda.

En total estimamos perder alrededor de 1,800 filas entre ítems vacíos y registros con nulos numéricos irre recuperables. Eso deja la capa final en aproximadamente 11,900 filas, suficientes para los análisis que vamos a hacer en la capa Gold.

4. Esquema objetivo: modelo dimensional

La capa final del ETL es un modelo estrella. Lo elegimos por dos razones: porque encaja bien con un proceso de negocio simple como las ventas, y porque permite aplicar directamente los conceptos de hechos, dimensiones, medidas y operaciones OLAP que se trabajaron en clase.

4.1 Decisiones de modelado

Optamos por un esquema estrella (no copo de nieve) porque la cardinalidad del dataset es muy baja: 8 categorías, 25 clientes, 200 ítems, 2 ubicaciones, 3 métodos de pago. No hay nada que justifique normalizar más allá de eso.

La granularidad de la tabla de hechos es una fila por transacción. Transaction ID es la llave natural y, como ya verificamos, es única en todo el dataset.

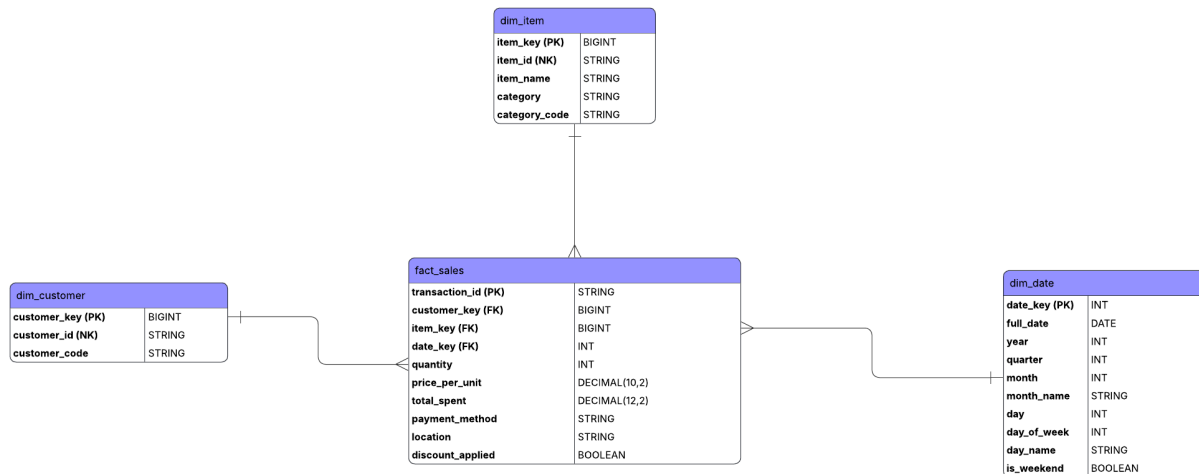
Cada dimensión tiene una llave surrogate (un BIGINT secuencial) como llave primaria, separada de la llave del negocio. Esto desacopla el modelo analítico de los identificadores operativos y deja la puerta abierta a manejar Slowly Changing Dimensions más adelante si hiciera falta.

Decidimos NO crear dimensiones para Payment Method ni Location. Tienen 3 y 2 valores respectivamente, y crear una dimensión completa con surrogate key, joins y todo, para algo tan chico, no aporta nada y solo agrega costo. Quedan como atributos degenerados directamente dentro de fact_sales. Lo mismo con Discount Applied.

Sobre la dimensión de productos: la categoría va dentro de dim_item, no en una dim_category aparte. La relación Item $\rightarrow$ Category es estable y no hay otros atributos de categoría que justifiquen separarla. Si en el futuro se quisiera agregar (por ejemplo, márgenes por categoría), se puede hacer sin reescribir el modelo.

4.2 Diagrama del modelo estrella

La tabla de hechos al centro, las tres dimensiones alrededor y los atributos degenerados dentro del propio fact_sales:



4.3 Definición de las tablas

fact_sales (tabla de hechos)

Una fila por transacción de venta. Aproximadamente 11,900 filas tras la limpieza.

Columna	Tipo	Llave	Descripción
transaction_id	STRING	PK	Identificador único de la transacción (formato TXN_#####).
customer_key	BIGINT	FK	Llave foránea hacia dim_customer.
item_key	BIGINT	FK	Llave foránea hacia dim_item.
date_key	INT	FK	Llave foránea hacia dim_date (formato AAAAMMDD).
quantity	INT	—	Medida aditiva. Cantidad de unidades vendidas.
price_per_unit	DECIMAL(10,2)	—	Medida no aditiva. Precio unitario al momento de la venta.
total_spent	DECIMAL(12,2)	—	Medida aditiva. Recalculado como quantity × price_per_unit.
payment_method	STRING	—	Atributo degenerado: Cash, Credit Card o Digital Wallet.
location	STRING	—	Atributo degenerado: In-store u Online.

Columna	Tipo	Llave	Descripción
discount_applied	BOOLEAN	—	Atributo degenerado. Booleano nativo tras la conversión.

dim_customer (clientes)

25 filas. Una por cada Customer ID único en el dataset.

Columna	Tipo	Llave	Descripción
customer_key	BIGINT	PK	Llave surrogate, secuencial.
Customer_id	STRING	NK	Llave natural del cliente (CUST_##).
Customer_code	STRING	—	Código numérico extraído del Customer ID (ej. 09).

Dim_item (productos)

200 filas: 25 ítems por cada una de las 8 categorías. La categoría se incluye acá adentro, sin dimensión separada.

Columna	Tipo	Llave	Descripción
item_key	BIGINT	PK	Llave surrogate, secuencial.
item_id	STRING	NK	Llave natural del ítem (Item_#_SUFIO).
item_name	STRING	—	Nombre del ítem.
category	STRING	—	Categoría del producto (8 valores).
category_code	STRING	—	Código de la categoría: BEV, BUT, CEA, EHE, FOOD, FUR, MILK, PAT.

dim_date (tiempo)

Aproximadamente 1,114 filas, una por cada día entre 2022-01-01 y 2025-01-18. La generamos programáticamente para que existan todas las fechas del rango, incluso aquellas en las que no hubo ninguna transacción.

Columna	Tipo	Llave	Descripción
date_key	INT	PK	Llave inteligente formato AAAAMMDD (ej. 20240408).
full_date	DATE	—	Fecha completa.

Columna	Tipo	Llave	Descripción
year	INT	—	Año (2022–2025).
quarter	INT	—	Trimestre (1–4).
month	INT	—	Mes (1–12).
month_name	STRING	—	Nombre del mes en español.
day	INT	—	Día del mes.
day_of_week	INT	—	Día de la semana (1=Lunes, 7=Domingo).
day_name	STRING	—	Nombre del día en español.
is_weekend	BOOLEAN	—	Verdadero si es sábado o domingo.

4.4 Medidas y consultas analíticas que el modelo soporta

La tabla de hechos tiene tres medidas. Quantity y Total Spent son aditivas: se pueden sumar a través de cualquier dimensión. Price Per Unit es no aditiva, así que solo permite promedios, mínimos o máximos. La medida principal para el negocio es Total Spent, porque representa el ingreso real.

Con este modelo se pueden responder consultas típicas del dominio retail: ingresos mensuales por categoría, evolución del ticket promedio por cliente, comparativa Online vs In-store, distribución de métodos de pago a lo largo del tiempo y análisis del impacto que tienen los descuentos sobre el volumen vendido.

Las jerarquías habilitan operaciones OLAP directas. La dimensión de tiempo permite drill-down de año a trimestre, mes y día, y roll-up en sentido inverso. La dimensión de productos permite navegar entre categoría e ítem. Los atributos degenerados (location, payment_method, discount_applied) habilitan slice y dice sin necesidad de joins adicionales.